Frontend Guide

For the three people building the student, teacher, and admin dashboards. You do not need Python, and you do not need a database.

Setup - five minutes

```
git clone <repo-url>
cd ai_tutor
cd frontend
npm install
```

Create `frontend/.env.local` with one line:

```
VITE_API_BASE=http://localhost:8000
```

When the backend owner starts their tunnel, replace that with the `https://something.trycloudflare.com` URL from the team channel. **The tunnel URL changes every time it restarts.** If the app suddenly cannot reach the backend, that is almost always why. Restart `npm run dev` after changing it.

Then:

```
npm run dev
```

Open http://localhost:5173

The stack

- **Vite + React + TypeScript** - `strict` is off on purpose. Types are documentation here, not a proof system. If TypeScript is fighting you, use `any` and move on.
- **Tailwind v4** - utility classes in `className`. No `tailwind.config.js` needed.
- **shadcn/ui** - copy-paste components that land in `src/components/ui/`.

Add a shadcn component:

```
cd frontend
npx shadcn@latest add button card badge table tabs dialog input
```

Folder rules

You own exactly one folder. Do not edit anyone else's.

```
frontend/src/
  pages/student/     person 1
  pages/teacher/     person 2
  pages/admin/       person 3
  pages/auth/        person 3
  components/        shared - agree in the channel before adding
  components/ui/     shadcn, safe for anyone to add to
  lib/api.ts         backend owner - do not edit
```

If you need a shared component, say so in the channel first. Two people creating `Card.tsx` at the same time is the classic way to lose an hour.

Calling the API

Every network call goes through `src/lib/api.ts`. Never write a bare `fetch` in a component.

```
import { api, User } from "@/lib/api";

const user = await api<User>("/auth/me");

const result = await api("/auth/login", {
  method: "POST",
  body: { email, password },
  auth: false,
});
```

`api()` already handles the token, JSON encoding, and turning an error response into a thrown `ApiError` with `.status`, `.code`, and `.message`.

```
import { ApiError } from "@/lib/api";

try {
  await api("/student/gaps");
} catch (e) {
  if (e instanceof ApiError && e.status === 401) {
    // send them to login
  }
}
```

Auth

After a successful login, store the token:

```
import { setToken, clearToken } from "@/lib/api";

const { token, user } = await api("/auth/login", {
  method: "POST", body: { email, password }, auth: false,
});
setToken(token);
```

The token is an opaque string. **Do not try to decode it** - it is not a JWT and there is nothing inside it. To know who is logged in, call `GET /auth/me`.

Log out with `clearToken()` plus `POST /auth/logout`.

Forgot password is **UI only**. Build the screen, collect the email, show a confirmation message, and make **no network call**. There is no backend route and there will not be one.

Working before the backend exists

Do not wait. Build against a fake that has the same shape as the contract.

```
// src/lib/mock.ts
export const MOCK_LESSON = {
  outcome: "answered",
  language: "en",
  body: "A vector can be split into perpendicular components...",
  citations: [
    { chunk_id: 1, material_id: 4, book_title: "Concepts of Physics, Vol 1",
      page_no: 143, chapter: "5. Newton's Laws", snippet: "..." },
  ],
```

```
  evidence: { alignment_percent: 82, sufficient: true, reason: null },
};
```

Then in your page:

```
const USE_MOCK = true;
const lesson = USE_MOCK ? MOCK_LESSON : await api(`/student/gaps/${id}/lesson`);
```

When the real endpoint lands, flip the flag. Nothing else changes, because the shape never changed.

The most important thing on this page

Every tutor response has an `outcome` field with **three** values, and all three come back as HTTP 200.

```
switch (res.outcome) {
  case "answered":
    return <Lesson body={res.body} citations={res.citations}
                   percent={res.evidence.alignment_percent} />;

  case "insufficient_evidence":
    return <NoEvidenceCard message={res.body} />;

  case "graded_work_refused":
    return <GuardrailCard message={res.body} hints={res.hints} />;
}
```

A refusal is a **correct** response, not an error. Those two refusal states are the features the judges care most about. If you only handle `answered`, they render as blank cards and the build looks broken exactly when it is working correctly.

TypeScript will help you here - `res.hints` only exists on the `graded_work_refused` branch, so the compiler tells you if you forgot to narrow.

Design tokens - so three dashboards look like one app

Use these and nothing else. Put them in `src/index.css` once (backend owner will merge the first version).

Purpose	Class
Page wrapper	mx-auto max-w-5xl p-6
Page title	text-2xl font-semibold
Section title	text-lg font-medium mt-8 mb-3
Muted text	text-sm opacity-70
Card	rounded-lg border p-4
Card grid	grid gap-4 md:grid-cols-2
Primary button	shadcn <Button>
Danger / refusal	border-amber-400 bg-amber-50
Good / high alignment	text-emerald-700
Low alignment	text-amber-700

Alignment badge, so all three dashboards render it identically:

```
function AlignmentBadge({ percent }: { percent: number }) {
  const tone = percent >= 70 ? "text-emerald-700" : "text-amber-700";
  return (
    <span className={`text-xs font-medium ${tone}`}>
      {percent}% syllabus aligned
    </span>
  );
}
```

Show Source

Every lesson card needs this. It is quoted directly in the problem statement, it is cheap, and judges look for it.

```
{citations.map((c) => (
  <div key={c.chunk_id} className="text-xs opacity-70">
    {c.book_title}, p.{c.page_no}
    {c.chapter && ` - ${c.chapter}`}
  </div>
))}
```

Accessibility - about an hour, worth a whole criterion

Read aloud, using the browser. No API, no cost:

```
function speak(text: string) {
  window.speechSynthesis.cancel();
  window.speechSynthesis.speak(new SpeechSynthesisUtterance(text));
}
```

Font size and contrast - toggle a class on `<html>`; the CSS already exists:

```
document.documentElement.classList.toggle("text-lg-mode");
document.documentElement.classList.toggle("high-contrast");
```

Also: every button must be reachable by Tab, every image needs `alt`, and never signal something with colour alone - put the number next to the colour.

Polling, not websockets

The teacher heatmap updates during the demo. Poll it. There are no websockets in this build.

```
useEffect(() => {
  const load = () => api("/teacher/misconceptions/heatmap?...").then(setData);
  load();
  const t = setInterval(load, 5000);
  return () => clearInterval(t);
}, []);
```

Before you push

```
npm run build
```

If that fails, `main` breaks for everyone. Fix it before pushing. Commit small and often, on your own branch.